

Học Sâu (Deep Learning)

Chương 15: Mô hình Ngôn ngữ & Kiến trúc Transformer

Đại học Đông Á

August 14, 2026

Nội dung chương

1. Mô hình Ngôn ngữ & Seq2Seq
2. Cơ chế Chú ý (Attention Mechanism)
3. Self-Attention và Query-Key-Value
4. Kiến trúc Transformer
5. Tổng kết

Language Model (Mô hình Ngôn ngữ) là gì?

- Xuyên suốt khóa học, ta đã quen với việc phân loại (Classification). Nhưng để máy tính có thể "nói chuyện" và "tạo ra văn bản", ta cần một cách tiếp cận khác.
- **Language Model:** Một mô hình thống kê học phân phối xác suất: $P(\text{token_tiếp_theo} | \text{các_token_trước_đó})$.
- Quá trình sinh văn bản (Text Generation): Dự đoán 1 token, ghép nó vào cuối chuỗi cũ, rồi lại đưa toàn bộ chuỗi mới vào mô hình để dự đoán token tiếp theo. Cứ lặp lại như vậy.

Chuẩn bị dữ liệu Sinh văn bản: Văn thơ Shakespeare

Để làm quen, ta sẽ dạy AI viết văn phong Shakespeare bằng cách đọc file text và cắt thành các chuỗi 100 ký tự.

```
1 import keras
2 import tensorflow as tf
3
4 # Download Shakespeare dataset
5 filename = keras.utils.get_file(
6     origin="https://storage.googleapis.com/download.tensorflow.org/data/shakespeare.
7     txt"
8 )
9
10 shakespeare = open(filename, "r").read()
11
12 sequence_length = 100
13
14 def split_input(text, seq_len):
15     for i in range(0, len(text), seq_len):
16         yield text[i : i + seq_len]
17
18 # Features: text[:-1]. Labels: text[1:] (Shifted by 1 character)
19 features = list(split_input(shakespeare[:-1], sequence_length))
20 labels = list(split_input(shakespeare[1:], sequence_length))
```

Xây dựng Bộ từ điển Cấp độ Ký tự (Char-level)

```
1 from keras import layers
2
3 # Map each character to an integer ID
4 tokenizer = layers.TextVectorization(
5     standardize=None,
6     split="character", # Split by char instead of word
7     output_sequence_length=sequence_length,
8 )
9 tokenizer.adapt(dataset.map(lambda text, labels: text))
10
11 print(tokenizer.vocabulary_size()) # Output: 67 unique chars
12
13 dataset = dataset.map(
14     lambda features, labels: (tokenizer(features), tokenizer(labels)),
15     num_parallel_calls=8,
16 )
17 training_data = dataset.shuffle(10000).batch(64).cache()
```

Xây dựng Mô hình Ngôn ngữ Thu nhỏ (GRU)

Mạng hồi quy GRU đọc chuỗi các ký tự và đưa ra xác suất 67 Class cho ký tự tiếp theo.

```
1 embedding_dim = 256
2 hidden_dim = 1024
3
4 inputs = layers.Input(shape=(sequence_length,), dtype="int", name="token_ids")
5 x = layers.Embedding(tokenizer.vocabulary_size(), embedding_dim)(inputs)
6 x = layers.GRU(hidden_dim, return_sequences=True)(x)
7 x = layers.Dropout(0.1)(x)
8
9 # Output probabilities for the 67 characters in our vocab
10 outputs = layers.Dense(tokenizer.vocabulary_size(), activation="softmax")(x)
11 model = keras.Model(inputs, outputs)
12
13 model.compile(
14     optimizer="adam",
15     loss="sparse_categorical_crossentropy",
16     metrics=["sparse_categorical_accuracy"],
17 )
18 model.fit(training_data, epochs=20)
```

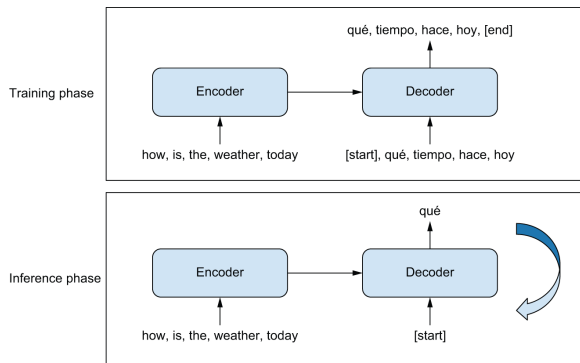
Vòng lặp Sinh ký tự (Sampling Loop)

Khác với Train, khi chạy thực tế (Inference), ta phải tạo một vòng lặp **Tự hồi quy** (Autoregressive): lấy output hiện tại đưa vào làm input cho bước tiếp theo.

```
1 import numpy as np
2
3 generated_ids = []
4 max_length = 250
5
6 # Generate characters one by one
7 for i in range(max_length):
8     # Predict probabilities for the next char
9     predictions, state = generation_model.predict((inputs, state), verbose=0)
10
11     # Pick the character with the highest probability
12     next_char = int(np.argmax(predictions, axis=-1)[0])
13     generated_ids.append(next_char)
14
15     # The output becomes the new input
16     inputs = keras.ops.expand_dims([next_char], axis=0)
```

Kiến trúc Sequence-to-Sequence (Dịch máy)

- Sequence-to-Sequence (Seq2Seq) là bài toán biến một chuỗi đầu vào (Tiếng Anh) thành một chuỗi đầu ra (Tiếng Tây Ban Nha).
- Không thể dịch từng từ một (Word-by-word) vì ngữ pháp hai ngôn ngữ hoàn toàn khác nhau. Phải đọc trọn vẹn cả câu rồi mới bắt đầu dịch.



Hình 15.1: Học chuỗi sang chuỗi (Seq2Seq)

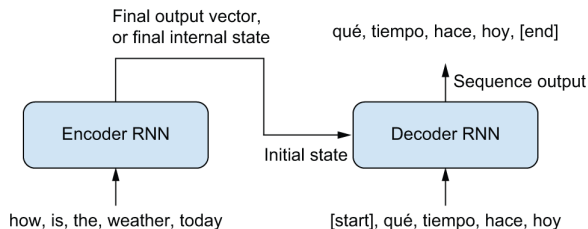
Chuẩn bị Dataset Đa đầu vào (Multi-Input Dataset)

Bài toán Dịch máy yêu cầu dữ liệu rất đặc biệt. Bộ Giải mã (Decoder) cần biết **Câu nguồn tiếng Anh** và **Câu đích tiếng Tây Ban Nha bị dịch đi 1 từ**.

```
1 def format_dataset(eng, spa):
2     eng = english_tokenizer(eng)
3     spa = spanish_tokenizer(spa)
4
5     # Features contain BOTH the English source AND the Spanish input shifted
6     features = {
7         "english": eng,
8         "spanish": spa[:, :-1] # Drop the last token
9     }
10
11    # The targets are the Spanish sentence shifted forward
12    labels = spa[:, 1:] # Drop the first token
13
14    # Tell Keras to ignore padding tokens (0) when computing Loss
15    sample_weights = labels != 0
16
17    return features, labels, sample_weights
```

Nút thắt cổ chai của Seq2Seq sử dụng RNN/LSTM

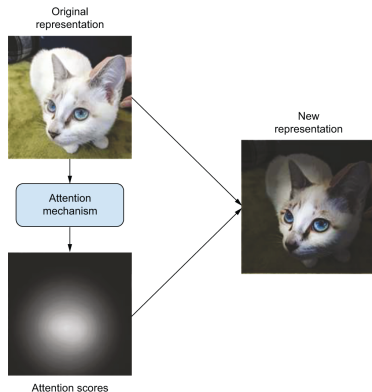
- **Encoder (Bộ mã hóa):** Đọc toàn bộ câu Tiếng Anh, nén toàn bộ ý nghĩa vào một véc-tơ Trạng thái duy nhất (Context Vector).
- **Decoder (Bộ giải mã):** Lấy Context Vector đó làm điểm tựa để nhả ra từng từ Tiếng Tây Ban Nha.
- **Khuyết điểm chí mạng:** Context Vector trở thành nút thắt cổ chai (Bottleneck). Nếu câu văn quá dài (50 từ), một véc-tơ duy nhất không thể chứa nổi toàn bộ ngữ nghĩa.



Hình 15.2: Nút thắt của Seq2Seq RNN

Sự ra đời của Cơ chế Attention

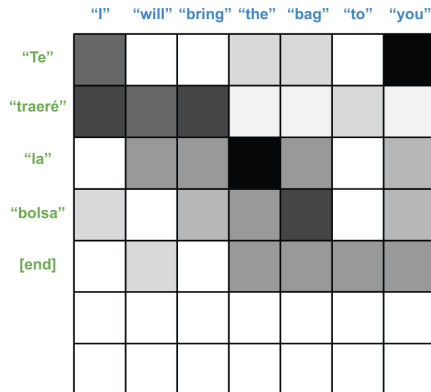
- Tại sao phải nén toàn bộ câu vào 1 véc-tơ duy nhất? Tại sao không cho Decoder "nhìn" lại toàn bộ chuỗi đầu vào ở mỗi bước dịch?
- **Cơ chế Attention (2014):** Khi Decoder sinh ra từ thứ t , nó sẽ tự động tính toán xem trong số tất cả các từ của Encoder, từ nào là **quan trọng nhất** ở thời điểm hiện tại để "chú ý" vào.



Hình 15.3: Ý tưởng của Cơ chế Attention

Điểm Chú ý (Attention Scores)

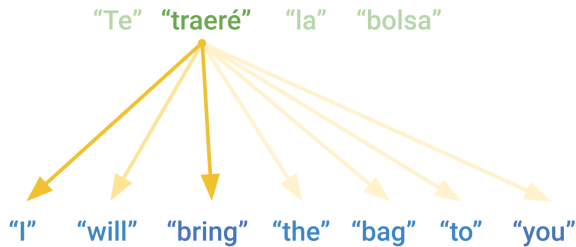
- Điểm chú ý (Attention Scores) là tập hợp các trọng số từ 0 đến 1 (tổng bằng 1).
- Ví dụ, khi Decoder đang dịch từ "station" (Trạm), nó sẽ gán điểm chú ý 0.95 cho từ "estación" trong câu gốc, và gán gần 0.0 cho các từ "I", "am", "at".
- Context Vector ở mỗi bước dịch giờ đây là Tích chập (Weighted Sum) của tất cả các trạng thái Encoder nhân với điểm chú ý.



Hình 15.4: Cách tính Toán học Attention Scores

Trực quan hóa Ma trận Attention

- Khi vẽ Ma trận điểm chú ý, ta thấy mô hình tự động học được cách "óng hàng" (Alignment) giữa 2 ngôn ngữ.
- Đặc biệt, nó xử lý hoàn hảo hiện tượng đảo ngữ pháp (VD: Tính từ đứng sau Danh từ trong tiếng Pháp/Tây Ban Nha).



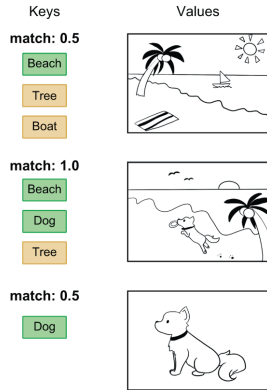
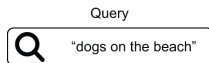
Hình 15.5: Biểu đồ phân bố Attention

Truy vấn Cơ sở dữ liệu: Query, Key, Value

Để tổng quát hóa Attention, các nhà khoa học đã mượn khái niệm từ Hệ quản trị Cơ sở dữ liệu:

- **Query (Q):** Truy vấn ta đang tìm kiếm (Từ hiện tại đang xét).
- **Key (K):** Nhãn của các mục trong CSDL (Các từ khác trong câu).
- **Value (V):** Nội dung thực sự của các mục đó.

Mô hình sẽ so khớp Q với tất cả các K . K nào càng giống Q thì V của nó sẽ càng đóng góp nhiều vào kết quả.



Hình 15.6: Cơ chế Query - Key - Value

Phương trình Scaled Dot-Product Attention

- Đây là phương trình Toán học quan trọng nhất thập kỷ của Trí tuệ Nhân tạo:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

- **Giải thích:**

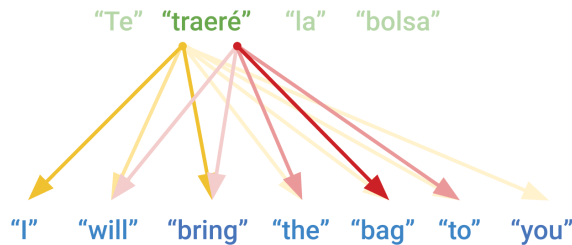
- QK^T : Tích vô hướng (Dot Product) đo độ tương đồng giữa Query và toàn bộ Key.
- $\sqrt{d_k}$: Hệ số chia (Scaled) nhằm ổn định đạo hàm không bị quá lớn.
- softmax : Ép các điểm tương đồng thành xác suất phân bố từ 0 đến 1.
- $\times V$: Nhân kết quả xác suất với các Giá trị thực tế.

Self-Attention (Tự Chú ý)

- Điều gì xảy ra nếu Q , K , và V đều đến từ **Cùng một chuỗi dữ liệu**?
- Đó gọi là **Self-Attention**. Nó cho phép một từ trong câu tự quan sát toàn bộ các từ còn lại trong chính câu đó để hiểu rõ ngữ cảnh của bản thân.
- Ví dụ câu: "The bank of the river" vs "The bank of America". Chữ "bank" sẽ tập trung sự chú ý vào "river" (bờ sông) hay "America" (ngân hàng) để thay đổi véc-tơ ngữ nghĩa của chính nó.

Multi-Head Attention (Chú ý Đa luồng)

- Thay vì chỉ tính Self-Attention 1 lần, ta tính nó h lần song song (gọi là h Heads).
- Tại sao? Vì một từ có rất nhiều khía cạnh: Ngữ pháp, Đại từ nhân xưng, Thời gian, Cảm xúc.
- Mỗi Head sẽ học cách "chú ý" vào một khía cạnh khác nhau của câu văn, làm cho không gian biểu diễn vô cùng phong phú.



Hình 15.7: Kiến trúc Multi-Head Attention

Cài đặt khối Transformer Encoder

Trong Transformer, không có LSTM. Chúng ta dựa hoàn toàn vào MultiHeadAttention và Dense. Kỹ thuật **Residual Connection** được dùng để tránh mất mát độ dốc.

```
1 class TransformerEncoder(keras.Layer):
2     def __init__(self, hidden_dim, intermediate_dim, num_heads):
3         super().__init__()
4         self.self_attention = layers.MultiHeadAttention(num_heads, hidden_dim //
5 num_heads)
6         self.self_attention_layernorm = layers.LayerNormalization()
7
8         self.dense_1 = layers.Dense(intermediate_dim, activation="relu")
9         self.dense_2 = layers.Dense(hidden_dim)
10        self.dense_layernorm = layers.LayerNormalization()
11
12    def call(self, source, source_mask):
13        # 1. Self Attention with Residual Add & Norm
14        residual = x = source
15        x = self.self_attention(query=x, key=x, value=x, attention_mask=source_mask)
16        x = self.self_attention_layernorm(x + residual)
17
18        # 2. Feedforward with Residual Add & Norm
19        residual = x
```

Sự khác biệt: Layer Normalization vs Batch Normalization

Trong Transformer, chúng ta không dùng BatchNormalization (chuyên cho ảnh) mà dùng LayerNormalization (chuyên cho chuỗi).

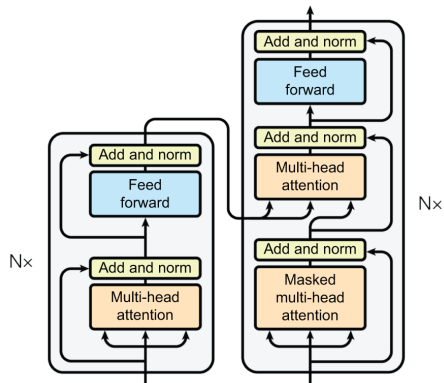
```
1 import numpy as np
2
3 # LayerNorm computes mean and variance for EACH sequence independently
4 def layer_normalization(batch_of_sequences):
5     mean = np.mean(batch_of_sequences, keepdims=True, axis=-1)
6     variance = np.var(batch_of_sequences, keepdims=True, axis=-1)
7     return (batch_of_sequences - mean) / variance
8
9 # BatchNorm computes mean and variance ACROSS the entire batch
10 def batch_normalization(batch_of_images):
11     mean = np.mean(batch_of_images, keepdims=True, axis=(0, 1, 2))
12     variance = np.var(batch_of_images, keepdims=True, axis=(0, 1, 2))
13     return (batch_of_images - mean) / variance
```

Bài báo vĩ đại: "Attention Is All You Need" (2017)

- Năm 2017, các nhà nghiên cứu Google tự hỏi: Nếu Attention mạnh mẽ như vậy, tại sao phải dùng RNN hay CNN nữa?
- Họ loại bỏ hoàn toàn RNN/CNN, chỉ thiết kế một mạng Nơ-ron cấu thành **hoàn toàn từ Multi-Head Attention**. Đó là **Transformer**.
- Ưu điểm tuyệt đối: Transformer có thể tính toán **Song song (Parallel)** trên GPU cho toàn bộ chuỗi cùng lúc, phá vỡ giới hạn tính tuần tự (từng từ một) chậm chạp của RNN.

Sơ đồ Kiến trúc Transformer

- Gồm 2 khối: Encoder (Bên trái) và Decoder (Bên phải).
- **Encoder:** Dùng Self-Attention để làm giàu ngữ cảnh cho câu đầu vào.
- **Decoder:** Dùng Causal Self-Attention (Che tương lai để không ăn gian) kết hợp với Cross-Attention (Lấy Q từ Decoder, K và V từ Encoder) để sinh từ tiếp theo.



Hình 15.8: Sơ đồ kinh điển của Transformer

Cài đặt khối Transformer Decoder

Decoder có tới 2 khối Attention. Khối đầu tiên là Causal Self-Attention. Khối thứ hai là Cross-Attention nhận Query từ Decoder và Key-Value từ Encoder.

```
1 class TransformerDecoder(keras.Layer):
2     def __init__(self, hidden_dim, intermediate_dim, num_heads):
3         super().__init__()
4         # Layers initialization omitted for brevity...
5
6     def call(self, target, source, source_mask):
7         # 1. Causal Self-Attention (Mask future words)
8         residual = x = target
9         x = self.self_attention(query=x, key=x, value=x, use_causal_mask=True)
10        x = self.self_attention_layernorm(x + residual)
11
12        # 2. Cross-Attention (Decoder attends to Encoder)
13        residual = x
14        x = self.cross_attention(query=x, key=source, value=source, attention_mask=
15        source_mask)
16        x = self.cross_attention_layernorm(x + residual)
17
18        # 3. Feedforward
19        residual = x
```

Mặt nạ Causal Masking là gì?

Trong `self_attention` của Decoder, ta dùng tham số `use_causal_mask=True`. Điều này nhằm che đi các từ "tương lai". Nếu không có mặt nạ này, mô hình sẽ "ăn gian" nhìn thấy đáp án ngay trong dữ liệu luyện tập. Ma trận Causal trông như sau:

```
1 [
2   [1, 0, 0, 0, 0], # Token 0 only sees token 0
3   [1, 1, 0, 0, 0], # Token 1 sees token 0, 1
4   [1, 1, 1, 0, 0], # Token 2 sees token 0, 1, 2
5   [1, 1, 1, 1, 0],
6   [1, 1, 1, 1, 1],
7 ]
```

Đây là cách Transformer giữ được tốc độ song song nhưng đảm bảo logic một chiều (như RNN).

Mảnh ghép cuối: Positional Encoding (Mã hóa Vị trí)

- Do tính toán song song, Transformer xử lý một câu giống như Bag-of-Words: Không hề biết từ nào đứng trước, từ nào đứng sau!
- Giải pháp: Cộng thêm một véc-tơ **Positional Encoding** (Dựa trên hàm Sine và Cosine hoặc Learnable Embeddings) vào véc-tơ Embedding của từ ban đầu.
- Nhờ đó, mỗi từ mang theo một "dấu vân tay" về vị trí của nó trong câu, giúp Transformer kết hợp tốc độ tính toán song song và khả năng hiểu ngữ pháp.

Cài đặt Lớp Positional Embedding

Tạo 2 Embedding: một cho Từ, một cho Vị trí (0, 1, 2...). Sau đó cộng chúng lại.

```
1 from keras import ops
2
3 class PositionalEmbedding(keras.Layer):
4     def __init__(self, sequence_length, input_dim, output_dim):
5         super().__init__()
6         self.token_embeddings = layers.Embedding(input_dim, output_dim)
7         self.position_embeddings = layers.Embedding(sequence_length, output_dim)
8
9     def call(self, inputs):
10        # Generate position indices: [0, 1, 2, ..., sequence_length-1]
11        positions = ops.cumsum(ops.ones_like(inputs), axis=-1) - 1
12
13        # Get embeddings
14        embedded_tokens = self.token_embeddings(inputs)
15        embedded_positions = self.position_embeddings(positions)
16
17        # Add them together
18        return embedded_tokens + embedded_positions
```

Ráp nối Kiến trúc Transformer Hoàn chỉnh

Gộp tất cả mọi thứ lại: Positional Embeddings → Encoder → Decoder → Dense.

```
1 source = keras.Input(shape=(None,), dtype="int32", name="english")
2 x = PositionalEmbedding(sequence_length, vocab_size, hidden_dim)(source)
3 encoder_output = TransformerEncoder(hidden_dim, intermediate_dim, num_heads)(
4     source=x, source_mask=source != 0
5 )
6
7 target = keras.Input(shape=(None,), dtype="int32", name="spanish")
8 x = PositionalEmbedding(sequence_length, vocab_size, hidden_dim)(target)
9 x = TransformerDecoder(hidden_dim, intermediate_dim, num_heads)(
10     target=x, source=encoder_output, source_mask=source != 0
11 )
12
13 x = layers.Dropout(0.5)(x)
14 target_predictions = layers.Dense(vocab_size, activation="softmax")(x)
15
16 transformer = keras.Model([source, target], target_predictions)
17 transformer.compile(optimizer="adam", loss="sparse_categorical_crossentropy")
```

Kết quả Dịch máy Anh - Tây Ban Nha

Sau khoảng 30 kỷ nguyên huấn luyện, Transformer đạt độ chính xác $>67\%$ (Cao hơn rất nhiều so với mô hình RNN cùng kích thước).

```
1 -  
2 The resemblance between these two men is uncanny.  
3 [start] el parecido entre estos cantantes de dos hombres son asombrosa [end]  
4 -  
5 I'll see you at the library tomorrow.  
6 [start] te vere en la biblioteca manana [end]  
7 -  
8 Do you know how to ride a bicycle?  
9 [start] sabes montar en bici [end]
```

Lưu ý: Mặc dù ngữ pháp có vài lỗi nhỏ, nhưng Transformer đã dịch chuẩn xác hầu hết các câu dài và ngữ cảnh phức tạp.

Tại sao Transformer lại vượt trội so với RNN?

- **Hiệu suất song song (Parallelism):** RNN cần phải duyệt qua mã thông báo thứ $N - 1$ mới có thể tính toán cho mã thông báo thứ N . Vòng lặp tuần tự này ngăn cản việc tận dụng sức mạnh tính toán song song của GPU.
- Transformer không có vòng lặp! Nó tính toán Attention cho **tất cả các mã thông báo cùng một lúc** thông qua phép nhân Ma trận. Điều này giúp tốc độ huấn luyện tăng lên gấp nhiều lần.
- **Ngữ cảnh tầm xa (Long-range context):** Trong RNN, thông tin ở đầu câu thường bị "quên" khi đến cuối câu. Với Self-Attention, mọi từ đều có kết nối trực tiếp $O(1)$ tới mọi từ khác trong câu, giải quyết hoàn hảo vấn đề giảm Gradient.

Mô hình Ngôn ngữ Che giấu (Masked Language Model)

- Thay vì đoán từ tiếp theo như GPT, BERT (và RoBERTa) sử dụng **Masked Language Model (MLM)**.
- Thuật toán sẽ che ngẫu nhiên 15% các từ trong câu bằng mã thông báo đặc biệt [MASK]. Sau đó, nó yêu cầu Transformer dùng các từ xung quanh để đoán xem từ bị che đi là gì.
- Ví dụ: "Con mèo đang ngủ trên chiếc [MASK] nhỏ." → Dự đoán: "giường".
- Nhờ cấu trúc Self-Attention hai chiều (Bidirectional), mô hình học được ngữ nghĩa của từ dựa trên toàn bộ các từ đứng trước và đứng sau nó.

Kỷ nguyên của Pre-trained Transformers khổng lồ

Mô hình Transformer dịch máy mà chúng ta vừa xây dựng có khoảng 14 triệu tham số. Các kiến trúc mạnh mẽ nhất trên thế giới hiện nay như BERT, RoBERTa hay GPT-4 có cấu trúc cốt lõi y hệt, nhưng được mở rộng lên tới **Hàng tỷ (Billions) tham số**.

Bạn hoàn toàn có thể sử dụng sức mạnh này bằng KerasHub:

```
1 import keras_hub
2
3 # Load the pretrained Tokenizer
4 tokenizer = keras_hub.models.Tokenizer.from_preset("roberta_base_en")
5
6 # Load the 124M parameter pretrained Transformer Backbone
7 backbone = keras_hub.models.Backbone.from_preset("roberta_base_en")
8
9 print(tokenizer("The quick brown fox"))
10 # Output: Array([133, 2119, 6219, 23602])
```

RoBERTa đã được "đọc" 160GB văn bản Tiếng Anh trên Wikipedia và Internet bằng hàng ngàn GPU. Nó tốn vài trăm ngàn Đô la chi phí tiền điện.

Tinh chỉnh (Fine-tuning) RoBERTa

Ta có thể cắm thêm một lớp Dense vào đuôi của RoBERTa Backbone để dạy nó thực hiện mọi bài toán NLP mình muốn, từ Phân loại Cảm xúc đến Tìm kiếm thông tin. Quá trình này diễn ra cực kỳ nhanh và chuẩn xác.

```
1 # Attach a Classification Head to RoBERTa
2 classifier = keras_hub.models.TextClassifier.from_preset(
3     "roberta_base_en",
4     num_classes=2,
5 )
6
7 classifier.compile(
8     loss=keras.losses.SparseCategoricalCrossentropy(from_logits=True),
9     optimizer="adam",
10    metrics=["accuracy"],
11 )
12
13 # Fine-tune on our small IMDb Dataset
14 classifier.fit(train_ds, validation_data=val_ds, epochs=1)
```

Tổng kết Chương 15

- **Sequence-to-Sequence:** Mô hình nền tảng cho Dịch máy, nhưng RNN vướng phải nút thắt cổ chai về véc-tơ ngữ cảnh.
- **Attention Mechanism:** Giải quyết bằng cách tính điểm trọng số cho toàn bộ câu đầu vào tại mỗi bước sinh từ.
- **Q-K-V Self-Attention:** Thuật toán tối thượng (Scaled Dot-Product) cho phép từ trong câu tự ánh xạ lên các từ khác để hiểu sâu ngữ cảnh.
- **Transformer:** Kiến trúc loại bỏ RNN, sử dụng hoàn toàn Multi-Head Attention. Đây chính là "trái kế" cấu thành nên các siêu AI như ChatGPT, GPT-4, BERT hiện nay.